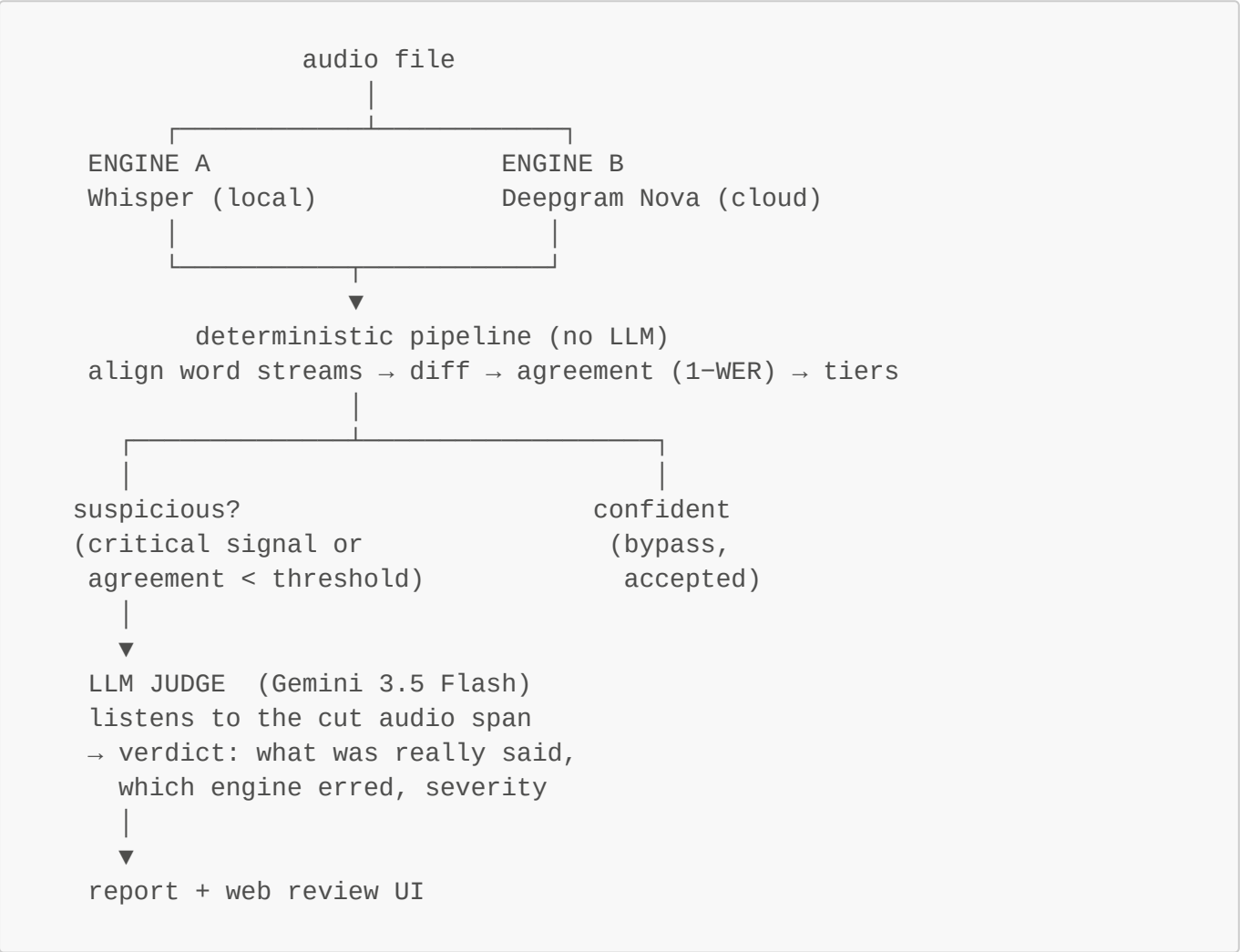
Approach 2 — two-engine consensus + audio-grounded LLM verification

No gold transcript required. Two independent STT engines transcribe the same audio; their word streams are aligned, every span is scored by how strongly the engines agree, and only the low-agreement / suspicious spans are sent to an audio-capable LLM that listens to the actual clip and arbitrates.

Architecture



Design rules:

- The engines are **peers** — neither is ground truth. Agreement means *low suspicion*, not *correctness*.
- The LLM **listens to the audio** and says what was actually spoken; it never just picks the more plausible transcript.
- Alignment, diffing, scoring, and agreement are **purely deterministic** — the LLM is only ever the second opinion on segments the detector flags.

The three stages

Stage	Command	Cost	What it does
1. Transcribe	`make transcribe`	Deepgram (cloud) + local Whisper	Both engines transcribe every file in `dataset/audio/`
2. Review	`make review`	free	Align, diff, score, tier, and sample — writes reports
3. Judge	`make judge`	Gemini (cloud)	LLM verifies the flagged segments and writes verdicts

`make run` runs all three, then opens the web UI.

Prerequisites

- Python 3.10+, `ffmpeg` on PATH
- `DEEPGRAM_API_KEY` — required for engine B (Deepgram Nova)
- `GEMINI_API_KEY` — required for the judge stage (Google AI Studio)
- Whisper runs locally via `faster-whisper` (model `small`, auto-downloaded)

Setup (one-time)

```
cd /path/to/audio_vs_transcript
make setup            # creates .venv, installs deps, checks ffmpeg + .env
```

`make` targets use the `.venv` automatically. To run the CLI directly, activate the virtualenv first:

```
source .venv/bin/activate        # bash/zsh (Windows:
.venv\Scripts\activate)
```

Keys live in the repo-root `.env` (auto-loaded):

```
DEEPGRAM_API_KEY=...
GEMINI_API_KEY=...
```

Run

One command for the whole pipeline:

```
make run
```

…or run the stages individually:

```
make transcribe                # stage 1: all files in dataset/audio/
make review                    # stage 2: reports for every file
make judge                     # stage 3: LLM verdicts for flagged segments
make ui                        # open the interactive review UI
make tests                     # run the test suite
```

Same pipeline at the CLI level (from the repo root, with `.venv` active):

```
python -m approach_2.main transcribe            # all files
python -m approach_2.main transcribe audio-3.ogg   # one file
python -m approach_2.main review                # all files
python -m approach_2.main review audio-1        # one file
python -m approach_2.main evaluate audio-1      # transcribe (if missing) +
review in one step
python -m approach_2.main judge audio-1         # + LLM-judge disagreements
uvicorn approach_2.api:app --port 8000          # review UI
```

Output

```
dataset/whisper/<name>.txt, .segments.json     stage 1 — engine A
dataset/deepgram/<name>.txt, .segments.json    stage 1 — engine B
dataset/review/<name>/report.{json,txt,md,srt,vtt}   stage 2 — aligned diff
+ scores
dataset/review/<name>/judgments.json           stage 3 — LLM verdicts
```

Each report segment carries: aligned word pairs (sub/ins/del), per-word confidence, agreement score, and a tier (`auto_accept` ≥ 98, `review_technical` 90–97, `mandatory` < 90).

An LLM verdict adds: `classification` (`missing | extra | hallucinated | incorrect | conflicting | accurate`), `correct_content`, `whisper_error`, `deepgram_error`, `severity` (`low | medium | high | critical`), `explanation`, `evidence`. Exports (txt/srt/vtt) use the corrected content when present.

Config (via `.env`)

Variable	Default	Meaning
`WHISPER_MODEL`	`small`	faster-whisper size (engine A)
`DEEPGRAM_API_KEY`	—	required for engine B (Deepgram Nova)
`DEEPGRAM_MODEL`	`nova-3`	Deepgram model
`GLOSSARY_PATH`	(empty)	optional domain-term wordlist (one per line)

Variable	Default	Meaning
`GEMINI_API_KEY`	—	required for the LLM judge stage
`GEMINI_MODEL`	`gemini-3.5-flash`	audio-capable judge model
`LLM_DISAGREE_THRESHOLD`	`0.9`	agreement below ⇒ LLM call
`REVIEW_DISAGREE_THRESHOLD`	`0.9`	agreement below ⇒ always human-reviewed
`JUDGE_PAD_SECONDS`	`0.5`	padding on each side of a cut audio span

The threshold

Agreement (`1 − WER`) is a score from **0.0 to 1.0** saying how closely the two engines' word streams match on a segment: `1.0` = identical, `0.5` = half the words differ.

The cutoff is **0.9** — any segment scoring **below 0.9** is considered a disagreement. Below that line the engines materially disagree, so the segment is judged (LLM) and reviewed (human). At or above it the engines essentially agree, and the segment is trusted.

There are **two separate 0.9 thresholds** because "spend money on an LLM call" and "cost a human their attention" are different decisions:

Config	Default	Meaning
`LLM_DISAGREE_THRESHOLD`	`0.9`	below ⇒ segment goes to the LLM judge
`REVIEW_DISAGREE_THRESHOLD`	`0.9`	below ⇒ segment is always human-reviewed

Lowering one does not lower the other. And note: the critical signals below fire **regardless** of the score, so a `cholecystectomy`/`colosyctomy` swap at agreement 0.97 is still flagged.

When is a segment sent to the LLM?

A segment is flagged iff **any** of these holds (deterministic, in `src/judge.py`):

- one engine missed it entirely (`engine_a`/`engine_b` is `None`);
- agreement < `LLM_DISAGREE_THRESHOLD` (0.9);
- a **critical signal** is present, regardless of the agreement score:
  - a negation changed ("requires" → "does not require");
  - a number differs ("20 mg" vs "200 mg");
  - a glossary/domain term changed (medication, anatomy, procedure);
  - a content word was added or removed;
  - a long technical word was substituted (likely a garbled domain term).

Short substitutions ("seattl"/"seattle", "spray"/"sprays") are treated as spelling/plural noise and left to the agreement threshold alone.

Categories

The deterministic detector (`src/judge.py::category()`) and the LLM verdict both report a category. The detector uses **the same vocabulary as Approach 1**, so the two approaches' outputs are directly comparable:

Category	Detector signal	LLM verdict
`match`	no critical signal	`accurate`
`missing_information`	one side dropped a word / had no text	`missing`
`incorrect_information`	number, glossary or technical word changed	`incorrect`
`conflicting_information`	negation flipped meaning	`conflicting`
`hallucinated_information`	one side added content not in the audio	`extra`, `hallucinated`

Tests

```
make tests
```

`tests/test_e2e.py` runs the whole path against the committed dataset transcripts (skipped automatically if they are not present).